Hindi and Nepali Small Language Models

Report on Phases 1–3 and the Bonus Experiment

Tanisha Upadhyay
CL3-410: Language Modeling and Applications

September 2026

1 Project Overview

In this project, I built two language models from scratch: Model H for Hindi and Model L for Nepali. Phase 1 covered data collection and tokenization, Phase 2 covered pretraining, and Phase 3 covered reasoning finetuning. I also trained a Hindi model without positional embeddings as a bonus experiment.

Each model had its own corpus, tokenizer, embeddings, weights, and checkpoints. Although both languages use Devanagari, they did not share any learned parameters.

Both models contain about 25 million parameters, but each processed more than 250 million training tokens. Tokens measure the text seen during training; parameters are the numerical values that the model learns. The same parameters are updated repeatedly as more text is processed. Additional epochs increase training exposure without increasing model size or unique corpus content.

Table 1: Training scale and model size. M denotes million.

	Hindi	Nepali
Phase 1 training tokens	332.25M	273.17M
Phase 2 tokens processed	332.60M	273.94M
Parameters	25.29M	24.87M

2 Language Selection and Project Evolution

I chose Hindi as the higher-resource language because public sources provide substantial encyclopedic, government, literary, and news text.

The initial lower-resource language was Bhojpuri. I collected Wikipedia, Wikisource, Maina, Bhojpuri News, and Anjoria, along with a downloaded corpus. Additional Bhojpuri–Hindi filtering was needed because script checks could not separate the two languages. This work included a tokenizer, pretraining, and downstream experiments, but the final decontaminated corpus contained only about 22.37 million normalized training tokens.

I investigated FineWeb-2 and HPLT for expansion. A FineWeb-2 shard contained about 9.82 million tokens before project-specific filtering and deduplication. This was a possible contribution rather than confirmed new training data.

Nepali was used for the final comparison because a much larger corpus could be assembled while meeting the manual-collection requirement. Bhojpuri remains part of the project history. This change showed that language selection depends on usable data, rather than published dataset size alone.

3 Phase 1: Data Collection and Tokenization

3.1 Sources and preprocessing

Hindi text came from Wikipedia, Wikisource, Wikibooks, the Press Information Bureau, and IndicCorp v2. Nepali text came from Wikipedia, Onlinekhabar, and the verified and unverified Nepali web subsets of Sangraha. Manual collection refers to direct extraction from original websites or APIs, not individual human transcription or review of every document. Records retained source URLs, collection methods, dates, and licence information.

Cleaning included Unicode normalization, whitespace cleanup, removal of repeated lines, and source-specific removal of templates, tables, navigation, timestamps, and references. Documents needed at least 200 characters and 30 words. Script and digit checks removed many unsuitable records, although script checks alone cannot confirm language identity.

Table 2: Final Phase 1 training data.

	Hindi	Nepali
Documents	2,237,951	704,658
Training tokens	332.25M	273.17M
Manually collected tokens	83.27M	62.11M
Manual token share	25.06%	22.74%

3.2 Deduplication and splits

I removed exact duplicate documents, repeated paragraphs, and near duplicates. Original-source copies were preferred when they overlapped with downloaded records. Nepali's document audit recorded 17,813 exact and 1,293 near duplicates removed; paragraph processing removed 135,681 repeated paragraphs.

Downloaded records were downsampled where necessary to meet the minimum 20% manual-token requirement. A 22% manual-word target provided a buffer because word share and token share differ. Both final corpora exceeded the required token share.

Documents were split deterministically into 98% training, 1% validation, and 1% test data. An audit found no exact Nepali document matches against Hindi or Bhojpuri. This does not rule out all near-duplicate or semantic overlap. A separate post-split Nepali near-document audit was not run.

3.3 Tokenizer selection

I trained separate SentencePiece Unigram tokenizers on the training splits and compared 16k, 24k, and 32k vocabularies using validation text. Numbers were split into individual digits to avoid storing many distinct multi-digit values.

Table 3: Selected tokenizer results.

	Hindi	Nepali
Vocabulary size	24,000	16,000
Tokens per word	1.2928	1.3522
Validation vocabulary utilization	83.93%	92.89%
Unknown-token rate	0.001487%	0.001359%

Hindi's 24k tokenizer reduced word splitting compared with 16k without the embedding cost of 32k. Nepali's 16k tokenizer had high vocabulary use and fewer rare pieces, leaving more parameters

for the Transformer. The main lesson was that larger vocabularies are not always better for small models: shorter sequences come at the cost of larger embeddings. Data quality and the manual-share requirement also limited the achieved corpus sizes.

4 Phase 2: Pretraining

4.1 Architecture and training

I implemented decoder-only Transformers in PyTorch rather than using pretrained models. Both had nine layers, six attention heads, width 384, and a 256-token context. They used causal attention, learned absolute positions, pre-layer normalization, GELU feed-forward layers, and tied input/output embeddings.

Causal masking prevents access to future tokens, while positional embeddings supply order information. Tied embeddings save parameters by reusing the vocabulary matrix. Nepali used wider feed-forward layers because its smaller vocabulary saved embedding parameters.

Table 4: Architecture and pretraining results.

	Hindi	Nepali
Layers / attention heads	9 / 6	9 / 6
Model width / context	384 / 256	384 / 256
Feed-forward width	1,536	1,920
Parameters	25,285,248	24,870,912
Optimizer steps	20,300	16,720
Validation perplexity	60.06	64.48
Test perplexity	39.57	48.85
Test bits per byte	0.5282	0.4377

Documents were packed with end-of-sequence separators. Training used AdamW, gradient clipping, 500 warmup steps, and cosine learning-rate decay from 3×10^{-4} to 3×10^{-5}. Gradient accumulation gave 16,384 tokens per update. The GPU runs used bfloat16, with resumable checkpoints for interrupted Kaggle sessions. Each model processed approximately one corpus pass.

Phase 1 token counts and Phase 2 packed-stream counts came from separately finalized artifacts; their differences cannot be explained by separators alone.

4.2 Evaluation and observations

Hindi had lower test perplexity, but Nepali had lower bits per byte. Perplexity depends on tokenization, so it cannot alone establish a cross-language ranking. Bits per byte provides a more comparable unit.

I tested greedy generation and temperatures 0.5, 1.0, and 1.5 on 50 fixed test prefixes. Greedy decoding often repeated phrases. Temperature 0.5 gave a useful balance between repetition and reference overlap; higher temperatures increased variety but sometimes weakened coherence. Single-reference overlap scores cannot capture every valid continuation.

Attention plots confirmed causal masking and often showed more concentrated attention in later layers on the selected prefix. They describe example behavior rather than prove what a head understands. Overall, pretraining learned useful language patterns but did not guarantee instruction following or coherent generation.

5 Phase 3: Reasoning Finetuning

5.1 Dataset and training

I generated native-language examples for direct comparison, largest/smallest selection, equality, and two-hop transitivity. Answers depended only on prompt facts and were checked symbolically.

Each language had 20,000 training examples, 2,000 validation examples, 2,000 test examples, and two 500-example challenges. The challenges used names or question wording absent from training. Duplicate worlds and prompts, unknown tokens, and overlength examples were rejected. The Nepali release had no cross-split world or prompt overlap and a maximum serialized length of 52 tokens.

Both models were finetuned for 2,500 updates with effective batch size 16. Loss applied only to answers and explanations. Training used AdamW, 100 warmup steps, and cosine decay from 10^{-4} to 10^{-5}. Nepali checkpoint selection used the full validation split; the historical Hindi run used a smaller subset, with full validation measured afterward.

5.2 Answer accuracy and explanation verification

The original evaluation allowed 16 new tokens. The explanation protocol allowed 96 and explicitly requested a reason. Since the instruction changed too, score differences cannot be attributed only to output length.

Answer accuracy checks the declared answer. Joint verification additionally requires a sufficient explanation supported by the facts. A correct name followed by a wrong comparison passes answer accuracy but fails joint verification. The checker evaluates the written explanation, not the model's internal reasoning.

Table 5: Primary-test results on 2,000 examples per language.

	Hindi	Nepali
Pretrained answer accuracy	0.00%	0.00%
Finetuned answer accuracy, 16 tokens	61.55%	56.30%
Finetuned answer accuracy, 96 tokens	86.20%	78.95%
Joint verification, 96 tokens	86.15%	78.55%

The pretrained models failed this response task, rather than proving an absence of all reasoning ability. Finetuning produced large gains. Under the explanation protocol, one correct Hindi answer and eight correct Nepali answers failed proof verification. The 16-token protocol produced no complete verified proofs, with truncation contributing to failure.

Table 6: Joint verification by task and challenge, using 96 tokens.

	Hindi	Nepali
Direct comparison	94.80%	86.20%
Largest/smallest selection	83.00%	69.00%
Equality	66.80%	59.00%
Two-hop transitivity	100.00%	100.00%
Unseen names	0.00%	0.00%
Unseen wording	64.60%	57.40%

Both models solved familiar-name transitivity, but equality and largest/smallest selection were harder. Transfer to new wording was partial; verified transfer to new names failed. Name familiarity and token splitting may contribute, but were not isolated experimentally.

5.3 Forgetting and sample-size experiments

After finetuning, generic Hindi perplexity rose from 39.57 to 628.12 and Nepali from 48.85 to 23,992.87. Bits per byte rose from 0.5282 to 0.9252 and from 0.4377 to 1.1352, respectively. This shows catastrophic forgetting: learning the narrow task damaged earlier language-modeling performance.

I also trained balanced 1k, 3k, 5k, 8k, and 12k subsets for two epochs each. The reasoning diagnostic used 100 fixed direct-comparison validation examples.

Table 7: Generic perplexity and narrow diagnostic accuracy.

Examples	Hindi PPL	Hindi accuracy	Nepali PPL	Nepali accuracy
0	39.57	0%	48.85	0%
1,000	55.30	49%	76.18	49%
3,000	68.89	52%	99.43	49%
5,000	87.38	40%	175.19	56%
8,000	158.50	52%	411.36	42%
12,000	333.86	41%	1,730.74	44%
20,000	628.12	85%	23,992.87	52%

Nepali's 5k run gave the best narrow diagnostic with much less forgetting than 20k. It cannot be called the best overall checkpoint without full task and challenge evaluation. Sample size also changed the number of updates.

On fixed prompts, Nepali's layer-8 attention entropy fell from 2.108 to 1.770 and distance rose from 9.639 to 12.196 tokens. Hindi changed less. These observations do not establish a causal reasoning mechanism.

6 Overall Understanding

The project showed that data preparation, vocabulary size, and architecture are closely connected. Larger vocabularies shorten sequences but use more parameters. More text is useful only when its quality and overlap are considered.

Pretraining learned language patterns, and finetuning improved controlled reasoning. However, success with familiar names did not guarantee transfer to new names. Reasoning improvement also came with a large loss of general-language ability, especially for Nepali.

The differences cannot be assigned to one cause: data scale, tokenization, capacity allocation, and training sensitivity all differed. A useful next step would be full evaluation of the smaller finetuning runs, followed by experiments mixing language-modeling data into finetuning or updating fewer parameters to reduce forgetting.

Bonus Experiment: Removing Positional Embeddings

I retrained Hindi without learned absolute positional embeddings, keeping the corpus, tokenizer, context, backbone, seed, optimizer, and token budget the same. Causal masking remained active. Removing the position table reduced the parameter count by 98,304 (0.39%), without adding capacity elsewhere.

The model completed 20,300 updates and processed 332,595,200 tokens. Validation perplexity increased from 60.06 to 62.46.

Test perplexity increased by 3.86%. Greedy and low-temperature outputs generally became more repetitive, although some overlap scores improved at temperature 1.0. Attention became sharper and more local on the fixed prefix.

Table 8: Hindi test results with and without positional embeddings.

	Standard	No positions
Cross-entropy	3.6782	3.7161
Perplexity	39.57	41.10
Bits per byte	0.5282	0.5336

Without positional embeddings, tokens lose their direct absolute-index signal. Order information is not removed entirely, because causal masking still gives different positions different preceding contexts. The model retained much of its performance, but the results support keeping positional embeddings in this setup. The conclusion is limited to this single experiment.